When Adversarial Prediction Still Compresses: Tokenization and Adversarial Robustness in LLM-Based Lossless Compression

Anonymous Author(s)
Affiliation
Address
email

Large language models (LLMs) can be powerful lossless compressors: their next-token probabilities coupled with entropy coding can reduce data below its original size. Yet existing evaluations largely single out predictive performance of the LLM, leaving an important systems question understudied: when does compression survive severe model mismatch? We show that the answer lies in treating LLM-based compression as a two-layer system of (1) tokenization as a stable, queryable representation and (2) predictive entropy coding as an optional cold-storage layer, guarded by a block-level selector that stores the smaller of Qwen+AC and Brotli. On text8, Qwen2.5-0.5B compresses to 17.6% of raw size, outperforming Qwen token IDs (41.3%) and the conventional codec Brotli (29.6%). Selecting the least-probable valid next token requires a prediction rate $5.68\times$ that of text8, measured in bits/token, yet long tokens buffer these errors and keep the complete data stream at 36.2%. Restricting candidates to one-byte tokens only removes this buffer and expands the complete stream to 122.8%, while conventional codecs remain compressive. We therefore introduce an objective that maximizes decoded-byte-normalized surprisal, attacking the buffer directly and producing an arithmetic payload of 122.4% and a complete stream of 144.3% of raw size.

1 Who Is Actually Doing the Heavy Lifting in LLM-Based Compression?

Strings are a core data type in production databases: in an Amazon Redshift fleet study, variable-length strings account for 52.1% of stored columns and 53.8% of predicate columns [1]. General-purpose systems must therefore deal with arbitrary inputs rather than only text that matches a model's training distribution. As data accumulates, compression becomes especially important for cold storage, where space savings must be weighed against encoding cost and access latency [2]. An LLM codec therefore poses system's challenges: it must preserve queryability (e.g. equality checks, joins, aggregations before decoding), handle any/arbitrary/unexpected input, and fail gracefully even when prediction is poor.

We study a layered architecture for this setting. The tokenizer maps strings to stable token IDs that act as a global dictionary and support operations over encoded data without model inference [3]; these IDs form the lossless, queryable representation. For cold storage or transport, an LLM predicts the IDs and an arithmetic coder compresses them. This predictive layer is optional: a block-level guard can store the smaller of Qwen+AC and a conventional codec such as Brotli [4], bounding learned-codec expansion by the fallback size.

Arithmetic coding assigns shorter lossless descriptions to more probable outcomes [5, 6]. Prior LLM codecs combine next-token probabilities with arithmetic coding or another lossless coding scheme [7]; tokenization itself can also act as lossless precompression [8], with byte-pair encoding (BPE) originating as a compression algorithm [9] and its effectiveness linked to shorter token sequences [10]. From a systems perspective, an LLM codec competes with traditional hand-engineered compression algorithms such as Brotli [4] and FSST [11].

What remains unclear is how much of an LLM codec's behavior comes from its token representation and how much comes from prediction. Delétang et al. compare ASCII and BPE [12] on natural

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

enwik9, but the two effects have not been isolated under inputs chosen against realized end-to-end size. This matters because prediction loss is measured in bits/token while compression is measured in bits/source byte: a poor probability model may still sit inside a pipeline that compresses when its tokens represent many bytes, whereas low-fertility tokenization can expose expansion. We therefore study the tokenizer–predictor–coder pipeline as a two-layer system.

Contributions. We (i) factor end-to-end compression rate into tokenizer fertility and predictive coding under natural, random, and worst-case inputs; (ii) formalize token- and byte-level objectives with realized-code accounting that reveal when tokenization absorbs predictive surprise and when the final stream expands; and (iii) derive a layered database design that preserves a queryable token representation while guarding optional predictive coding with a conventional block-level fallback.

2 Compression Objectives and How to Construct Worst-Case Inputs

Our central distinction is between prediction difficulty and end-to-end compression failure. A token can be extremely unlikely under the language model while decoding to few source bytes. We use one notation throughout: let $x = (x_1, \ldots, x_N)$ be the generated token sequence, let $h_t = x_{<t}$ be its token history, let $\mathrm{D} = \mathrm{decode}$ denote the decoder, and let $p_\theta(v \mid h)$ be the language model probability of token v after history h. For a sequence of N tokens, let $|\mathrm{decode}(x)|_{\mathrm{UTF8}}$ denote the number of source bytes obtained after decoding the token sequence x. For example, if $\mathrm{decode}(x) = \texttt{hello}$, then $|\mathrm{decode}(x)|_{\mathrm{UTF8}} = 5$ bytes. Let $B(x)$ be shorthand for the decoded source size in bytes: $B(x) = |\mathrm{decode}(x)|_{\mathrm{UTF8}}$.

The token-level objective, MinProb, selects the least-probable admissible continuation $v^\star_{\mathrm{tok}} \in G(h)$:

$$v^\star_{\mathrm{tok}} = \arg\min_{v \in G(h)} p_\theta(v \mid h) = \arg\max_{v \in G(h)} \{-\log_2 p_\theta(v \mid h)\}. \tag{1}$$

This is the LLM-centric token-level worst case. It maximizes model surprisal, ideal coded bits/token, but not necessarily bits/source byte. To account for variable-length tokenization, define the incremental decoded byte length contributed by the token v,

$$\Delta b(v; h) = |\mathrm{decode}(h \parallel v)|_{\mathrm{UTF8}} - |\mathrm{decode}(h)|_{\mathrm{UTF8}}. \tag{2}$$

The byte-aware MaxSurprisal/Byte objective instead selects $v^\star_{\mathrm{byte}}$:

$$v^\star_{\mathrm{byte}} = \arg\max_{v \in G} \frac{-\log_2 p_\theta(v \mid h)}{\Delta b(v; h)} \qquad [\mathrm{bits/byte}]. \tag{3}$$

Unlike Eq. 1, it directly targets the ideal entropy-coding cost in bits/newly decoded source byte. For a complete sequence x, the cumulative ideal model cost is the sequence NLL $L_\theta(x)$

$$L_\theta(x) = \sum_{t=1}^{N} \ell_\theta(x_t \mid x_{<t}) = -\sum_{t=1}^{N} \log_2 p_\theta(x_t \mid x_{<t}) \qquad [\mathrm{bits}], \tag{4}$$

where any fixed initial context is implicit in $x_{<1}$. For serialized output size $S_{\mathrm{codec}}(x) = |\mathrm{SerializeCodec}(x)|$ in bytes, the ideal and realized source-normalized size ratios are

$$R_{\mathrm{ideal}}(x) = 100\,\frac{L_\theta(x)}{8B(x)}\ [\%] \qquad R_{\mathrm{real}}(x) = 100\,\frac{S_{\mathrm{codec}}(x)}{B(x)}\ [\%]. \tag{5}$$

Values below 100% denote compression; R_{real} additionally captures arithmetic-coder, metadata, and framing effects.

2.1 Prediction Loss Is Not Compression Loss

We define tokenizer fertility as the average decoded source-byte count:

$$f(x) = \frac{|\mathrm{decode}(x)|_{\mathrm{UTF8}}}{N} \qquad [\mathrm{bytes/token}]., \tag{6}$$

Using the sequence NLL $L_\theta(x)$, the ideal predictive rate is

$$R_{\text{ideal}}(x) = 100\,\frac{L_\theta(x)}{8|\text{decode}(x)|_{\text{UTF8}}} = \underbrace{\frac{L_\theta(x)}{N}}_{\text{bits/token}}\,\underbrace{\frac{N}{|\text{decode}(x)|_{\text{UTF8}}}}_{\text{tokens/byte}}\,\frac{100}{8}. \tag{7}$$

Thus high bits/token can be offset by high bytes/token. Tokenization can buffer severe prediction failure when adversarial tokens represent many source bytes. Hence, there is a compression boundary (ignoring finite coder state, metadata overhead, etc.) that occurs at $\frac{L_\theta(x)}{N} < 8\,\frac{|\text{decode}(x)|_{\text{UTF8}}}{N} = 8f(x)$. A token representing f source bytes can tolerate up to $8f$ bits of surprisal before the predictive layer expands the source.

2.2 Token- and Byte-Level Attacks

Threat Model. We assume white-box access to the compression pipeline: the adversary can inspect the tokenizer and model logits when constructing an input and construct length-N adversarial inputs by autoregressive search over a candidate alphabet G using different scoring functions. To ensure that generated token sequences survive decoding and retokenization, every prefix must satisfy $T(\text{decode}(x_{1:t})) = x_{1:t}$; see Appendix 5.2.

Adversarial Input Generation. We benchmark two adversarial generation policies for against natural text from Wikipedia and random strings: *Minimum probability* ($v^\star_{\text{tok}}$) greedily maximizes Eq. 1, while *maximum surprisal/byte* ($v^\star_{\text{byte}}$) greedily maximizes Eq. 3. In addition, we provide a *one-byte UTF-8 ablation* restricting the candidate alphabet G to canonical tokens representing exactly one decoded byte, thereby removing variation in token fertility. Under this constraint, the multi-byte buffering is removed and the $v^\star_{\text{tok}}$ and $v^\star_{\text{byte}}$ objectives induce the same greedy ranking.

Practical Relevance (should be rewritten and the paragraph renamed). Together, these inputs model worst-case blocks caused by either benign model mismatch or malicious users. Among predictive objectives, MaxSurprisal/Byte ($v^\star_{\text{byte}}$) captures the relevant failure mode for end-to-end size; guards should therefore monitor realized bytes/source byte, not token-level loss alone. Together, these inputs model adverse blocks arising from either benign model mismatch or malicious users. Because token-level loss alone does not determine serialized size, a practical guard should monitor realized bytes per source byte.

3 Evaluation

3.1 Experimental Setup

We evaluate Qwen2.5-0.5B [13] on natural text, random text input, full-vocabulary adversarial sequences, and a one-byte UTF-8 ablation. Each input in the primary attack evaluation contains 10,000 tokens, except for the full-vocabulary MaxSurprisal/Byte ($v^\star_{\text{byte}}$) evaluation, which contains 1,024 tokens due to computational constraints. [1] All attacks use the same model, context, and arithmetic-coding configuration, and generated prefixes must round-trip through tokenizer decoding and re-encoding.

3.2 Results

Natural-to-Random Shift Removes the Predictive Advantage. As was to be expected, Qwen+AC strongly compresses natural text, but on random printable text its size ratio rises to 97.4%, compared with 83.0% for Brotli. Prediction still repairs much of the Qwen token IDs baseline's inefficiency, but under this distribution shift it no longer beats a conventional codec. It is worth noting that random input alone is still not able to break the 100% compression barrier for Qwen+AC.

Prediction Failure Does Not Imply Compression Failure. The MinProb ($v^\star_{\text{tok}}$) attack increases prediction difficulty for the LLM $5.68\times$, yet remains compressive because fertility is high at 7.09

[1] This shorter evaluation does not change the qualitative conclusion; see Appendix 5.3.

Table 1: **Source-normalized predictive coding under increasingly adverse inputs.** Token-level surprisal alone is not predictive of compression failure. ↓ lower is better, ↑ higher is better for compression. AC size R_{AC} reports the arithmetic-coder payload relative to raw UTF-8. † Final 1,024-token evaluation (one run).

Input / condition	Pred. cost ↓ (bits/token)	Fertility ↑ (bytes/token)	Ideal rate ↓ (bits/byte)	AC rate ↓ (bits/byte)	AC size ↓ (% raw)
Baselines					
Natural text (text8)	5.11	5.59	0.92	1.13	14.1%
Random canonical	8.78	1.31	6.71	7.58	94.8%
Adversaries					
MinProb ($v^{\star}_{\mathrm{tok}}$)	29.04 ± 0.04	7.09 ± 0.04	4.10 ± 0.02	2.53 ± 0.01	$31.7 \pm 0.2\%$
MaxSurprisal/Byte ($v^{\star}_{\mathrm{byte}}$)†	15.52	1.47	10.53	9.79	122.4%
Tokenizer ablation					
One-byte UTF-8	8.33 ± 0.00	1.00 ± 0.00	8.33 ± 0.00	9.57 ± 0.00	$119.6 \pm 0.0\%$

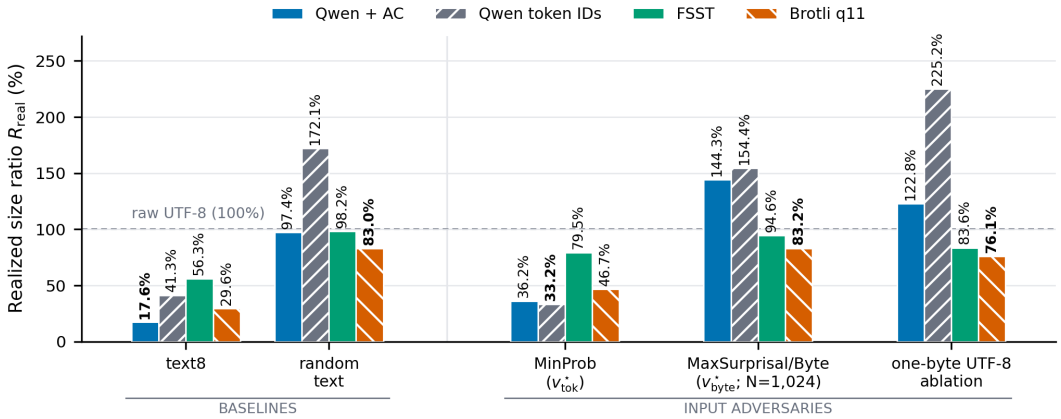

Figure 1: **End-to-end compression across natural vs. adversarial inputs.** Realized size ratio R_{real} for Qwen+AC, Qwen token IDs, FSST [11], and Brotli. Bold values mark the smallest stream for each input. All streams are round-trip-verified. Qwen model weights, tokenizer table are excluded.

source bytes/token. The Qwen token IDs baseline is slightly smaller than Qwen+AC on the same bytes, confirming that robustness comes from the multi-byte token representation rather than an additional predictive-coding gain. For Qwen token IDs, random input is larger than MinProb input, underscoring that tokenization alone provides substantial precompression.

Attacking Fertility-Adjusted Cost Causes Expansion. Restricting MinProb to a one-byte token vocabulary is one way of removing the fertility buffer and makes the complete Qwen+AC stream expand to 122.8%. MaxSurprisal/Byte ($v^{\star}_{\mathrm{byte}}$) attacks the same buffer directly over the full vocabulary and likewise causes expansion and is the most effective joint attack on prediction + tokenization. MaxSurprisal/Byte ($v^{\star}_{\mathrm{byte}}$) is the most effective attack on the complete Qwen + AC codec.

4 Implications and Conclusion

LLM-based compression survives model errors. LLM compression is an end-to-end systems property, not merely a property of the language model: tokenization, prediction, coding, and serialization jointly determine final size. On natural text (text8), the complete Qwen+AC reaches 17.6% of raw size. MinProb ($v^{\star}_{\mathrm{tok}}$) raises prediction cost to $5.68\times$ its text8 level, yet its payload remains at 31.7% because multi-byte tokens absorb the surprise; the Qwen token IDs baseline is slightly smaller than Qwen+AC on the same MinProb input. Conversely, restricting candidates to one-byte tokens removes the tokenization-buffer and makes the complete stream expand to 122.8%. Compression therefore survives poor prediction because of tokenizer fertility, not necessarily because the model predicts well.

Queryable token representation as the foundation. Token IDs provide a stable, queryable representation for hot data, while predictive coding can be reserved for sufficiently cold blocks. This separation matters because LLM compression trades storage savings for inference cost and access latency, making access frequency and retention time central to whether it is worthwhile [2]. A conventional codec such as Brotli can serve as a fallback, ensuring that predictive compression remains an optional optimization rather than a required representation.

Block-level guard. On eviction of cold data, the system can encode eligible blocks with Qwen+AC and Brotli and store the smaller complete stream with a one-bit codec tag. Selection should use realized bytes per source byte rather than token loss to bound storage expansion by the Brotli fallback. On access, Qwen+AC restores token IDs directly, while Brotli output must be retokenized before token-table execution (Figure 3).

References

[1] Alexander van Renen, Dominik Horn, Pascal Pfeil, Kapil Vaidya, Wenjian Dong, Murali Narayanaswamy, Zhengchun Liu, Gaurav Saxena, Andreas Kipf, and Tim Kraska. Why TPC is not enough: An analysis of the amazon redshift fleet. *Proceedings of the VLDB Endowment*, 17 (11):3694–3706, 2024. doi: 10.14778/3681954.3682031.

[2] Andreas Kipf et al. Waiting to decompress: The economics of llm-based compression. In *CIDR*, 2026.

[3] Tobias Schmidt et al. Accelerating string-heavy queries with LLM token tables. *PVLDB*, 19, 2026.

[4] Jyrki Alakuijala et al. Brotli compressed data format. RFC 7932, Internet Engineering Task Force, July 2016. URL `https://www.rfc-editor.org/info/rfc7932`.

[5] Ian H. Witten et al. Arithmetic coding for data compression. *Commun. ACM*, 30(6):520–540, June 1987. doi: 10.1145/214762.214771.

[6] Christian Steinruecken. *Lossless Data Compression*. PhD thesis, University of Cambridge, Cambridge, UK, 2015.

[7] Chandra Shekhara Kaushik Valmeekam et al. LLMZip: Lossless text compression using large language models. *arXiv preprint arXiv:2306.04050*, 2023.

[8] Grégoire Delétang et al. Language modeling is compression. In *ICLR*, 2024.

[9] Philip Gage. A new algorithm for data compression. *The C Users Journal*, 12(2):23–38, 1994.

[10] Matthias Gallé. Investigating the effectiveness of BPE: The power of shorter sequences. In *EMNLP-IJCNLP*, pages 1375–1381, Hong Kong, China, 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1141.

[11] Peter Boncz et al. FSST: Fast random access string compression. *Proceedings of the VLDB Endowment*, 13(11):2649–2661, 2020. doi: 10.14778/3407790.3407851.

[12] Rico Sennrich et al. Neural machine translation of rare words with subword units. In *ACL*, pages 1715–1725, Berlin, Germany, 2016. Association for Computational Linguistics. doi: 10.18653/v1/P16-1162.

[13] Qwen et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. doi: 10.48550/arXiv.2412.15115. URL `https://arxiv.org/abs/2412.15115`.

5 Appendix

5.1 Metric Definitions

The primary and auxiliary evaluations share the same token- and source-normalized quantities but differ in stream accounting:

$$\text{Bits/token} = \frac{L_\theta(x)}{N}, \tag{8}$$

$$\text{Bytes/token} = \frac{|\text{decode}(x)|_{\text{UTF8}}}{N}, \tag{9}$$

$$\text{Ideal bits/byte} = \frac{L_\theta(x)}{|\text{decode}(x)|_{\text{UTF8}}}, \tag{10}$$

$$\text{AC bits/byte} = \frac{S_{\text{AC}}(x)}{|\text{decode}(x)|_{\text{UTF8}}}, \tag{11}$$

$$R_{\text{AC}} = 100\,\frac{S_{\text{AC}}(x)}{8\,|\text{decode}(x)|_{\text{UTF8}}}, \tag{12}$$

$$R_{\text{real}} = 100\,\frac{|\text{SerializeCodec}(x)|}{|\text{decode}(x)|_{\text{UTF8}}} \quad [\%]. \tag{13}$$

Both ratios are relative to raw UTF-8; $R < 100\%$ denotes compression and $R > 100\%$ denotes expansion.

5.2 Evaluation Protocol

Canonical Inputs. Not every sequence of valid token IDs can arise from tokenizing its own decoded bytes. For example, suppose the vocabulary contains tokens for `a`, `b`, and `ab`. The token sequence $[\texttt{a}, \texttt{b}]$ decodes to `ab`, but the tokenizer may encode `ab` as the single token $[\texttt{ab}]$. Hence, decoding and retokenizing changes the token sequence. Since our attacks select token IDs directly from the model distribution, allowing such sequences would optimize over token histories that cannot occur for the corresponding source input. We therefore require every generated partial sequence to be canonical, i.e., $T(\text{decode}(x_{1:t})) = x_{1:t}$. This ensures that final decoding recovers exactly the same token IDs and bytes.

Model and Coder. We chose Qwen2.5-0.5B because its small size makes full-vocabulary adversarial scoring tractable while retaining a large vocabulary, and allows the experiments to run locally on a CPU without requiring GPU access. We use 151,643 non-special candidates, a 1,000-token context with 100 tokens retained between coding blocks, and KV caching. Arithmetic coding uses a 32-bit state and frequency total 262,144. The version-1 Qwen+AC stream records the seed, model and coder configuration, token and raw-byte counts, arithmetic payload, any required token dictionary, and framing. Model weights and the shared tokenizer table are shared decoder state and excluded from both Qwen representations.

Inputs and Aggregation. The primary evaluation uses 10,000-token sequences for natural text, random canonical input, MinProb ($v^\star_{\text{tok}}$), and the one-byte UTF-8 ablation. MaxSurprisal/Byte ($v^\star_{\text{byte}}$) is reported at the largest feasible budget of 1,024 tokens and is the final evaluation for that attack. Values marked with $\pm$ are means and sample standard deviations over three independent canonical seed runs; the MaxSurprisal/Byte ($v^\star_{\text{byte}}$) result is a single run.

Round-Trip Validation and Baselines. All generated prefixes satisfy

$$\text{T}(\text{D}(x_{1:t})) = x_{1:t}, \qquad t = 1, \ldots, N,$$

and every complete stream is independently decoded back to identical token IDs or UTF-8 bytes, as applicable. FSST uses the official implementation at commit `e638d4cf8c26129d73c242a4127b42b975de5b63`; Brotli uses quality 11; the Qwen token IDs baseline stores each ID in a fixed 18-bit representation with a stream header.

5.3 Detailed Results and Accounting

Effect of the Shorter MaxSurprisal/Byte ($v^{\star}_{\text{byte}}$) Sequence. The shorter sequence affects the magnitude but not the qualitative conclusion. Under the same replay configuration, the arithmetic-payload ratio is 136.4%, 133.2%, 131.7%, and 122.4% at $N = 32$, 64, 128, and $1{,}024$, respectively; even the longest evaluated prefix remains larger than raw UTF-8. The complete 1,024-token Qwen+AC stream is 144.3% of its 1,508 source bytes. Moreover, all bars in the MaxSurprisal/Byte group use those same source bytes, so their comparison is size matched. Fixed codec overhead is relatively larger on short inputs, which disadvantages rather than favors the conventional codecs.

Codec Comparison. Figure 1 and Table 2 compare codecs on identical source bytes. The four non-Max groups contain 9,996–13,087 bytes; the 1,024-token MaxSurprisal/Byte output contains 1,508 bytes. All plotted results are complete, round-trip-verified streams.

Stream Accounting. The auxiliary comparison reports both the Qwen+AC payload and complete stream, whose difference is 331–334 bytes across the four non-Max serialized inputs. For text8, this overhead raises the size ratio from a 14.3% arithmetic-coder payload to a 17.6% complete stream. The complete size is $S_{\text{total}} = S_{\text{payload}} + S_{\text{dict}} + S_{\text{frame}}$. Finite frequencies clip extremely small probabilities, so realized arithmetic length can be below unquantized NLL for MinProb; we report this as quantization, not improved prediction.

Tokenizer Fertility.

5.4 Beam Search Experiments

Search Setup. A realized-size beam search approximately maximizes Eq. 5 by cloning the codec state for candidate continuations. Unlike the greedy objectives, it captures arithmetic-coder, metadata, and framing effects during generation. Because the search considers only a bounded beam and candidate set, its result is an approximation to the sequence-level optimum.

Results. [TODO: Report the beam width, branch factor, token and byte budgets, search runtime, and realized-size results; compare against the greedy MinProb and MaxSurprisal/Byte attacks and the conventional-codec baselines.]

5.5 Guarded Compression Pipeline

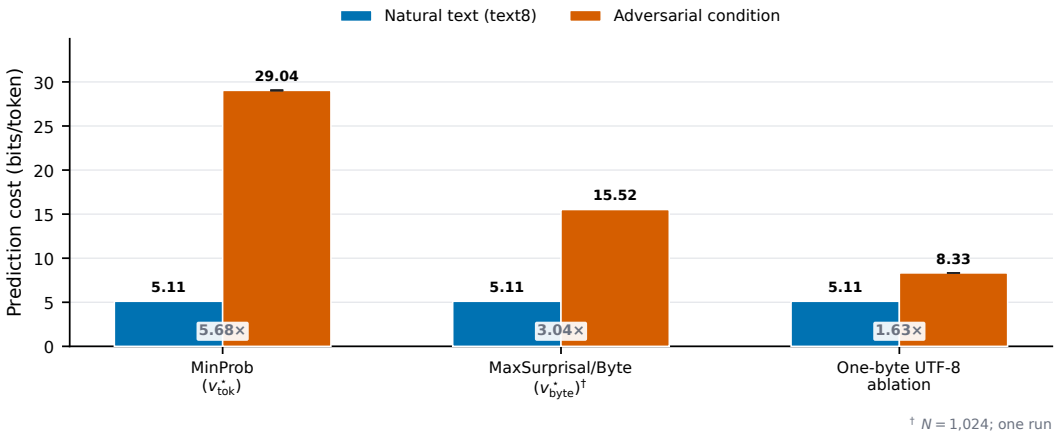

Figure 2: **Adversarial prediction difficulty.** Each condition is compared with the same natural-text prediction cost of 5.11 bits/token. Labels inside the plot report the ratio to text8. Error bars show sample standard deviation across three canonical seeds where available; † marks the final 1,024-token MaxSurprisal/Byte ($v^{\star}_{\text{byte}}$) single run.

Table 2: **Realized size ratio by input and codec.** Values are encoded bytes divided by raw UTF-8 bytes and reported as percentages; lower is better. AC payload excludes the Qwen container and is shown only for cost attribution. All entries in the four codec columns are complete, round-trip-verified streams, and bold marks the smallest complete stream per row.

Input	AC payload	Qwen+AC	Qwen token IDs	FSST	Brotli
text8	14.3%	17.6%	41.3%	56.3%	29.6%
Random printable	94.8%	97.4%	172.1%	98.2%	83.0%
MinProb ($v^{\star}_{\text{tok}}$) adversary	32.9%	36.2%	33.2%	79.5%	46.7%
MaxSurprisal/Byte ($v^{\star}_{\text{byte}}$; $N = 1{,}024$)	122.4%	144.3%	154.4%	94.6%	83.2%
One-byte UTF-8 ablation	119.5%	122.8%	225.2%	83.6%	76.1%

Table 3: Tokenizer fertility on the held-out 5 MB text8 test region (855,233 whitespace-delimited words). The text8 BPE is trained only on the first 90 MB with a 32k vocabulary. Fertility is tokens/word (lower is better); bytes/token is the compression-oriented inverse (higher is better). All tokenizers reconstruct the input exactly. The domain-trained BPE produces 4.4% fewer tokens than Qwen.

Tokenizer	Vocab.	Used	Tokens	Tok./word ↓	Bytes/tok. ↑
ASCII byte	128	27	5,000,000	5.8464	1.0000
text8 BPE (32k)	32,000	26,617	933,499	**1.0915**	**5.3562**
Qwen2.5	151,665	25,798	976,828	1.1422	5.1186

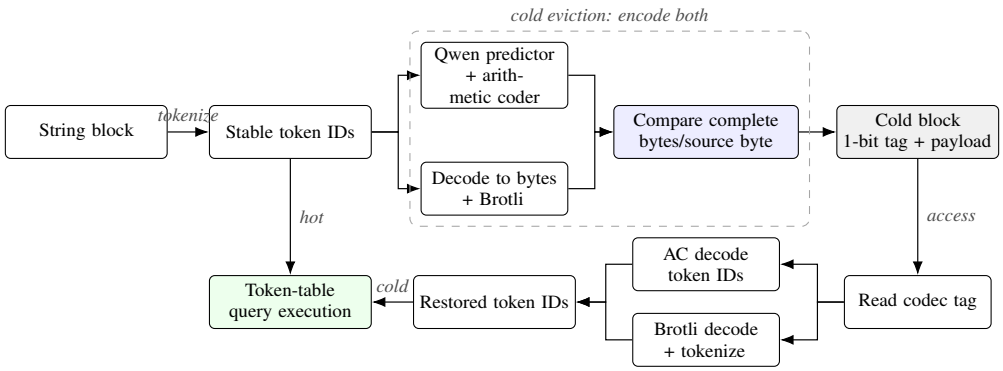

Figure 3: **Guarded layered compression pipeline.** Hot blocks retain queryable token IDs. At cold eviction, both complete encodings are produced and the smaller is stored with a codec tag. On access, either path restores token IDs before token-table query execution.